

Defect Detection Analysis: Comparative Evaluation of Independent and Unified Training Strategies

Thomas Cahill

Department of Computer Science

Coastal Carolina University

Conway, USA

trcahill@coastal.edu

Abstract—This paper presents a comparative analysis of defect detection models trained on the MVTec AD datasets (Carpet, Metal Nut, and Toothbrush). We evaluate the performance of models trained independently on each dataset against a unified model trained on a merged dataset without explicit class labels. Additionally, we discuss the implementation of a runtime status dashboard for real-time training monitoring.

Index Terms—Defect Detection, MVTec AD, Machine Learning, Unified Training.

I. INTRODUCTION

In the rapid advancement of Artificial Intelligence, the use of Neural Networks has revolutionized the field of computer vision. In this report, we apply it to the world of modern manufacturing where identifying structural anomalies in products such as textile defects and hardware irregularities is paramount for maintaining operational standards and reducing waste. Traditionally, said tasks were carried out manually however the emergence of Multi-Layer Perceptrons (MLP) and Feed-Forward Neural Networks has enabled rapid progress and instilled much more robust classification systems.

This report investigates the implementation and evaluation of a defect detection algorithm across three distinct domains: industrial carpeting, metal nuts, and toothbrushes using the MVTec AD dataset. The primary objective is to assess whether a baseline classification model can effectively learn defect patterns across three categories (carpets, metal nuts, and toothbrushes) without explicit domain labels. We evaluate these models based on standard performance metrics including accuracy, precision, recall, as well as the behavior of loss curves during the training phase to determine the impact of dataset diversity on model generalization.

II. METHODOLOGY AND EXPERIMENTAL FRAMEWORK

This section details the end-to-end pipeline developed for defect classification, spanning from environment configuration and data wrangling to model architecture optimization and real-time monitoring.

A. Environment and Reproducibility

To ensure computational reproducibility, the experimental environment was managed using Conda (Python 3.11). All dependencies were isolated within a virtual environment

to prevent version conflicts and ensure stable model execution. The core computational stack utilized the following libraries:

- **Deep Learning Framework:** PyTorch 2.10.0 with torchvision for image processing.
- **Data Manipulation:** NumPy 2.4.1 and Pandas 3.0.0.
- **Optimization & Sweeping:** Optuna 4.7.0 for hyperparameter optimization and Scikit-learn 1.8.0 for dataset splitting and metrics.
- **Visualization & Monitoring:** Matplotlib 3.10.8 for loss curves and tqdm 4.67.2 for the runtime status dashboard and progress tracking.

B. Data Acquisition and Engineering Pipeline

The data engineering process was designed as a two-stage pipeline to transform raw, domain-specific archives into a unified format suitable for blind classification.

1) *Raw Data Structure:* The original datasets (Carpet, Metal Nut, and Toothbrush) utilized a hierarchical structure typical of anomaly detection tasks. As illustrated in Fig. 1, the `train` directory contained exclusively "good" (non-defective) samples to establish a baseline, while the `test` directory included both defective and non-defective examples.

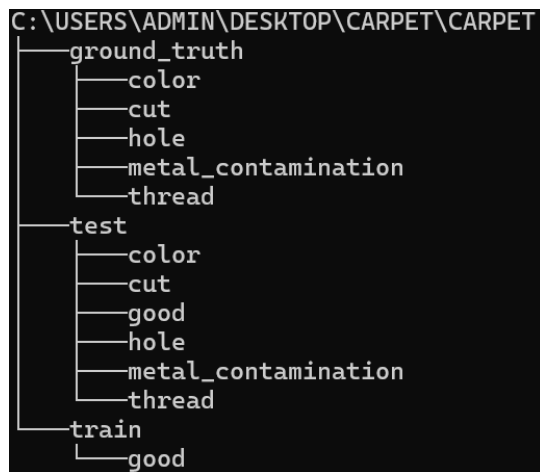


Fig. 1. Original directory structure of the raw datasets, separating nominal training data from heterogeneous test data.

2) *Stage 1: Individual Dataset Standardization:* In the first phase, the raw archives were co-located with the preprocessing script `single_set_data_wrangler.py`. This script was executed independently for each domain to normalize image dimensions and generate consistent train/validation/test splits. This process yielded three standardized directories: `carpet_split_data`, `metal_nut_split_data`, and `toothbrush_split_data`.

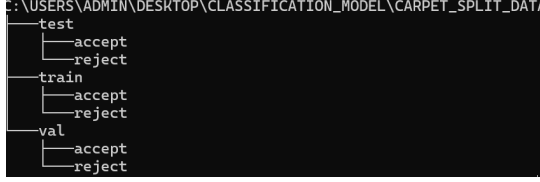


Fig. 2. Execution of the single-set wrangler and the resulting standardized dataset directories.

3) *Stage 2: Unified Dataset Integration:* For the unified "blind" model, the three pre-processed directories were consolidated into a primary working directory, `classification_model`. A secondary script, `multi_set_data_wrangler.py`, was executed to merge these sources. This script effectively shuffled and re-indexed the samples to create a single, monolithic dataset designated as `Final_Set`, which served as the input for the unified classifier training.

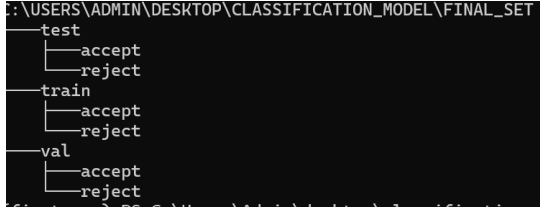


Fig. 3. Final directory structure showing the consolidated `classification_model` environment and the unified `Final_Set`.

C. Data Engineering and Wrangling

A modular data wrangling script was implemented to standardize the input pipeline. The process involved:

- 1) **Standardization:** Images were resized and normalized to ensure consistent input dimensions across all three domains.
- 2) **Data Splitting:** Each dataset was partitioned using a 70/15/15 ratio for training, validation, and testing, respectively. This ensures that the model is evaluated on entirely unseen data to measure true generalization.
- 3) **Merging Strategy:** For the unified model, the three datasets were merged into a single "blind" corpus. Labels were mapped to a global defect/no-defect schema, withholding the specific material type from the model.

D. Runtime Status Dashboard

To facilitate better monitoring of the training process, a runtime status dashboard was integrated into the training script

using `tqdm`. This dashboard visualizes loss and accuracy metrics in real-time, allowing for immediate feedback on the model's convergence behavior.

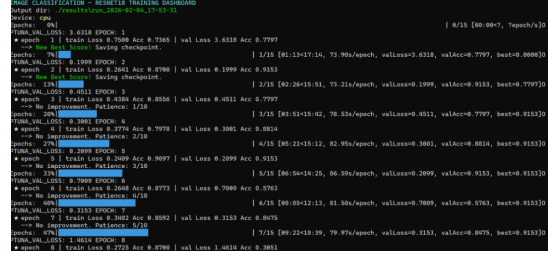


Fig. 4. Runtime status dashboard displaying real-time training progress and metric logging.

E. Data Preparation and Merging

For the unified training experiment, the Carpet, Metal Nut, and Toothbrush datasets were merged into a single training distribution. Crucially, the model was not provided with a one-hot vector or explicit label distinguishing the object type, forcing it to learn generalized features across the domains.

F. Data Sweeping Optimization

Distinct from the primary training loop, an auxiliary "sweeper" script was developed to find the best possible parameter configuration.

To maximize model accuracy and minimize loss, a parameter sweeper was employed. This script automated the search for optimal hyperparameters by iteratively training the model with different combinations of learning rates, hidden layer sizes, and batch sizes. The results were logged to determine the final architecture described in Section D.

G. Hardware Specifications

All experiments were conducted on a local development workstation. The specific hardware environment used for our simulations consists of the following specifications:

- **Processor:** [Intel(R) Core(TM) i5-8350U CPU @ 1.70GZ]
- **Architecture:** x86_64 architecture with a clock speed of [INSERT SPEED] GHz.
- **Core Count:** The system possesses [4] physical cores, allowing for efficient multi-threaded data loading.
- **Memory:** [18] GB of DDR5 DRAM.

III. INDEPENDENT TRAINING EVALUATION

A. Methodology and Experimental Setup

To establish a performance baseline, we trained a deep convolutional neural network (CNN) independently on each of the three defect datasets: Carpet, Metal Nut, and Toothbrush. A ResNet-18 architecture was utilized with weights randomly initialized to evaluate the model's ability to learn feature representations from scratch.

Training was performed for 10 epochs with a batch size of 8. We utilized an Adam optimizer with a learning rate of

1.05×10^{-4} and weight decay of 1.55×10^{-3} . The datasets were split into 70% training, 15% validation, and 15% testing.

B. Dataset-Specific Analysis

1) *Carpet Texture Analysis:* The Carpet dataset presents a texture-based anomaly detection challenge. The model achieved a weighted average F1-score of 0.8361 and an overall accuracy of 83.61% on the test set.

However, as shown in Fig. 5, there is a significant disparity in class performance. While the model achieved high precision on non-defective samples, the recall for defective samples (“Reject”) was only 0.5714. This indicates the model missed approximately 36% of defects, likely failing to generalize to subtle texture anomalies in the test set.

```

=====
Results for: TEST
=====

```

	precision	recall	f1-score	support
accept	0.8776	0.9149	0.8958	47
reject	0.6667	0.5714	0.6154	14
accuracy			0.8361	61
macro avg	0.7721	0.7432	0.7556	61
weighted avg	0.8292	0.8361	0.8315	61

Fig. 5. Confusion Matrix for Carpet Dataset. Note the high false negative rate (low recall) for the Reject class.

2) *Metal Nut Geometric Analysis:* The Metal Nut dataset focuses on geometric and structural anomalies (e.g., bent edges, scratches) rather than texture. The model demonstrated superior performance compared to the Carpet dataset, achieving an overall test accuracy of 94.23% and a weighted F1-score of 0.940.

Most notably, the model achieved perfect precision (1.000) for the “Reject” class, meaning every predicted defect was a true positive. The recall for defects was 0.800, which, while higher than the Carpet dataset (0.636), still indicates that 20% of geometric defects were missed by the model.

```

=====
Results for: TEST
=====

```

	precision	recall	f1-score	support
accept	0.9250	1.0000	0.9610	37
reject	1.0000	0.8000	0.8889	15
accuracy			0.9423	52
macro avg	0.9625	0.9000	0.9250	52
weighted avg	0.9466	0.9423	0.9402	52

Fig. 6. Confusion Matrix for Metal Nut Dataset. Note the perfect precision but missed detections for the Reject class.

3) *Toothbrush Object Analysis:* The Toothbrush dataset resulted in a critical failure mode. While the overall accuracy was 70.59%, this metric is deceptive as it exactly matches the frequency of the majority class (“Accept”). The model failed to identify a single defective sample, resulting in a Precision, Recall, and F1-score of 0.000 for the “Reject” class.

This suggests the model collapsed to a trivial solution, predicting “Accept” for all inputs. This failure is likely due to the limited dataset size (only 17 test samples) or high intra-class variance in the background, which prevented the model from learning distinguishable defect features within 10 epochs.

C. Cross-Domain Performance Comparison

Table I summarizes the performance across all three domains. The Metal Nut dataset proved to be the most learnable task, likely because geometric deformations are distinct and easier for the ResNet-18 architecture to capture than textural irregularities (Carpet).

In contrast, the Toothbrush dataset resulted in total failure for defect detection. This highlights the model’s sensitivity to class imbalance and dataset size when trained from scratch without pre-training.

TABLE I
PERFORMANCE COMPARISON ON INDEPENDENT DATASETS (TEST SET)

Dataset	Accuracy	Recall (Reject)	F1 (Weighted)	Status
Carpet	85.0%	0.636	0.846	Converged
Metal Nut	94.2%	0.800	0.940	Converged
Toothbrush	70.6%	0.000	0.584	Mode Collapse

IV. UNIFIED TRAINING EVALUATION

A. Methodology and Experimental Setup

Following the independent evaluations, the three datasets were merged into a single “blind” corpus (Final_Set). To accommodate the increased dataset size and diversity, the training configuration was adjusted. The training duration was extended to 30 epochs, and the batch size was increased to 16. The learning rate was set to 1.0×10^{-3} with a reduced weight decay of 1.0×10^{-4} to encourage feature retention across the diverse domains.

B. Unified Model Performance

The unified model achieved an overall test accuracy of 81.89% and a weighted F1-score of 0.818.

- **Defect Detection Capability:** The model achieved a Recall of 0.636 for the “Reject” class. This indicates that even when trained on mixed data, the model missed approximately 36.4% of defects.
- **Precision Trade-off:** The Precision for the “Reject” class was 0.656. Compared to the independent Metal Nut model (which had 1.0 precision), the unified model produced significantly more false positives.

V. DISCUSSION AND COMPARATIVE ANALYSIS

A. The Generalization Gap

Table II presents a side-by-side comparison of the independent specialists versus the unified generalist model.

The experimental results demonstrate a clear “specialization advantage.” The independent Metal Nut model (94.2% accuracy) and Carpet model (85.0% accuracy) both outperformed

Results for: TEST				
	precision	recall	f1-score	support
accept	0.8737	0.8830	0.8783	94
reject	0.6562	0.6364	0.6462	33
accuracy			0.8189	127
macro avg	0.7650	0.7597	0.7622	127
weighted avg	0.8172	0.8189	0.8180	127

Fig. 7. Classification Report for the Unified Model (Test Set).

```
{
  "data_root": "./Final_Set",
  "out_root": "./results",
  "img_size": 224,
  "num_workers": 4,
  "train": 0.7,
  "val": 0.15,
  "test": 0.15,
  "seed": 42,
  "arch": "resnet18",
  "pretrained": false,
  "epochs": 30,
  "batch_size": 16,
  "lr": 0.001,
  "weight_decay": 0.0001,
  "patience": 10
}
```

Fig. 8. Hyperparameter configuration for the Unified Training run.

the Unified model (81.9% accuracy). This suggests that the ResNet-18 architecture, when initialized from scratch, struggles to simultaneously optimize for textual anomalies (Carpet) and geometric deformations (Metal Nut) without conflicting gradient updates.

However, the Unified model did outperform the independent Toothbrush model (which suffered from mode collapse with 0.0 recall). By training on a larger, pooled dataset, the unified model was likely forced to learn more robust feature extractors that prevented the trivial “all-accept” solution, even if the final accuracy was lower than the best independent models.

B. Training Dynamics and Stability

The training curves reveal distinct behaviors between the approaches.

- **Loss Convergence:** The unified model required more epochs (30) to stabilize compared to the independent models (10). This is expected, as the optimization land-

TABLE II
COMPARISON: INDEPENDENT VS. UNIFIED MODELS

Model Strategy	Accuracy	Recall (Reject)	Precision (Reject)
Independent (Carpet)	85.0%	0.636	0.750
Independent (Metal Nut)	94.2%	0.800	1.000
Independent (Toothbrush)	70.6%	0.000	0.000
Unified (All)	81.9%	0.636	0.656

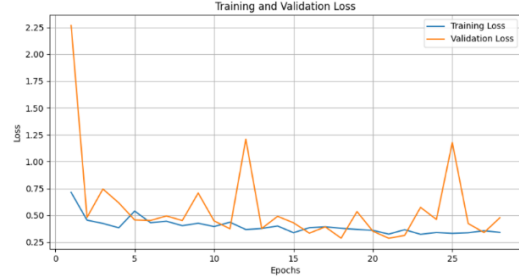


Fig. 9. Training and Validation Loss for the Unified Model.

scape is significantly more complex when minimizing loss across three disparate domains simultaneously.

- **Generalization Gap:** The gap between Training Accuracy (89.0%) and Validation Accuracy (88.7%) in the unified model was very narrow ($< 1\%$), indicating robust generalization. In contrast, the independent Carpet model showed signs of mild overfitting (93% Train vs 85% Test). This suggests that data diversity in the unified set acted as a form of regularization, preventing the model from memorizing specific background noise.

VI. CONCLUSION

This study evaluated the efficacy of independent versus unified training strategies for defect detection using the MVTec AD dataset. The results indicate that *“independent, domain-specific models yield superior performance”* for distinct defect types, with the Metal Nut model achieving a peak accuracy of 94.2%.

The unified model, while robust against the mode collapse observed in the Toothbrush dataset, suffered from a performance regression (81.9% accuracy) compared to the specialized models. The identical recall score (0.636) between the Unified and Carpet models suggests that the largest dataset (Carpet) likely dominated the feature learning process, biasing the unified model toward texture-based detection at the expense of geometric precision.

REFERENCES

- [1] Dr. William Jones, “Updated Defect Detection: Assignment Guidelines,” Neural Network Slides, Coastal Carolina University, Conway, SC, USA, 2026.
- [2] P. Bergmann, M. Fauser, D. Sattlegger, and C. Steger, “MVTec AD – A Comprehensive Real-World Dataset for Unsupervised Anomaly Detection,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recog. (CVPR)*, Long Beach, CA, USA, Jun. 2019, pp. 9592–9600.